

Examination Portal

File-to-File Flow Documentation

Arrows show which file calls/imports/depends on which other file.

1. Authentication Flow

From File	Calls / Uses	To File	What is passed
LoginComponent.jsx	onSubmit prop →	Login.jsx (page)	form { username, password }
Login.jsx	calls login()	AuthContext.jsx	username (or email), password
AuthContext.jsx	POST /auth/login	api.js → AuthController.java	{ username, password }
Register.jsx	calls register()	AuthContext.jsx	registration form data
AuthContext.jsx	POST /auth/register	api.js → AuthController.java	{ username, email, password, role, firstName, lastName }
AuthController.java	calls login()/register()	AuthService.java	LoginRequest / RegisterRequest DTO
AuthService.java	findByUsername().or(findByEmail())	UserRepository.java	username or email string
AuthService.java	authenticate()	AuthenticationManager → CustomUserDetailsService.java	username + raw password
AuthService.java	generateToken()	JwtUtil.java	username, role
AuthController.java	returns	AuthContext.jsx (frontend)	AuthResponse { token, id, username, email, role }
AuthContext.jsx	stores token	localStorage + api.js interceptor	JWT string

Security Filter Chain (every request)

From File	Calls / Uses	To File	What
Any HTTP request	→ intercepted by	JwtRequestFilter.java	extracts Bearer token from Authorization header
JwtRequestFilter.java	validateToken()	JwtUtil.java	token string → valid bool + username
JwtRequestFilter.java	loadUserByUsername()	CustomUserDetailsService.java	username → UserDetails
CustomUserDetailsService.java	findByUsername().or(findByEmail())	UserRepository.java	username/email → User entity
JwtRequestFilter.java	sets context	SecurityContextHolder	UsernamePasswordAuthenticationToken
SecurityConfig.java	configures rules	JwtRequestFilter.java	filter order + URL authorization rules

2. Admin Flow

2.1 Test Creation with Inline Questions

From File	Calls / Uses	To File	What is passed
TestManagementComponent.jsx	onSubmit prop →	TestManagement.jsx	form submit event
TestManagement.jsx	computes marksPerQuestion	(internal)	totalMarks ÷ inlineQuestions.length → passed as prop to component
TestManagement.jsx	POST /admin/tests via	api.js	CreateTestRequest { title, subjectId, durationMinutes, totalMarks, passingMarks, questions: [QuestionDTO] }
api.js	→ HTTP POST	AdminController.java /admin/tests	JSON body + Bearer token
AdminController.java	getUserByUsername()	UserService.java	username from JWT
AdminController.java	createTest(request, adminId)	TestService.java	CreateTestRequest + admin user ID
TestService.java	findById(subjectId)	SubjectRepository.java	subjectId → Subject entity
TestService.java	findById(userId)	UserRepository.java	userId → User entity
TestService.java	builds Question entities	Question.java (model)	sets test, subject, marks, difficulty per QuestionDTO
TestService.java	save(test)	TestRepository.java → MySQL	Test + all Questions in one transaction (CASCADE)

2.2 Admin Dashboard Stats

From File	Calls / Uses	To File	What
AdminDashboard.jsx	GET /admin/users, /admin/subjects, /admin/tests (parallel)	api.js → AdminController.java	Three parallel requests
AdminDashboard.jsx	derives totalQuestions	(internal — no API call)	sum of test.questionCount across all tests

2.3 Subject / User Management

From File	Calls / Uses	To File	What
SubjectManagement.jsx	GET POST PUT DELETE via api.js	AdminController.java /admin/subjects	Subject payloads
AdminController.java	delegates to	SubjectService.java	CRUD operations
SubjectService.java	queries	SubjectRepository.java → MySQL	subjects table

From File	Calls / Uses	To File	What
UserManagement.jsx	GET PUT DELETE via api.js	AdminController.java /admin/users	User payloads
AdminController.java	delegates to	UserService.java	CRUD operations
UserService.java	queries	UserRepository.java → MySQL	users table

3. Student Flow

3.1 Available Tests Page

From File	Calls / Uses	To File	What is passed
AvailableTests.jsx	GET /student/tests + GET /student/exams/attempts (parallel)	api.js → StudentController.java	Bearer token (both requests in Promise.all)
StudentController.java /student/tests	getActiveTests()	TestService.java	returns active tests with questionCount
StudentController.java /student/exams/attempts	getCompletedAttempts(studentId)	ExamService.java	returns completed ExamAttempt list
AvailableTests.jsx	builds completedTestIds Set	(internal)	Set of test IDs the student has already completed
AvailableTests.jsx	passes completedTestIds prop	AvailableTestsComponent.jsx	controls badge (Active/Completed) and button (Start Test / Already Attempted)
AvailableTestsComponent.jsx	reads test.questionCount	Test JSON response	server-computed field — not from questions array

3.2 Start Exam

From File	Calls / Uses	To File	What is passed
AvailableTestsComponent.jsx	onStartTest(testId) prop →	AvailableTests.jsx	testId (only for non-completed tests)
AvailableTests.jsx	POST /student/exams/start/{testId}	api.js → StudentController.java	testId + Bearer token
StudentController.java	startExam(studentId, testId)	ExamService.java	student user ID + testId
ExamService.java	save(attempt)	ExamAttemptRepository.java → MySQL	new ExamAttempt (status=IN_PROGRESS)
AvailableTests.jsx	navigate to exam page	ExamPage.jsx	{ state: { attemptId } }

3.3 Load Questions & Submit Answers

From File	Calls / Uses	To File	What is passed
ExamPage.jsx	GET /student/exams/{attemptId}/questions	api.js → StudentController.java	attemptId
StudentController.java	attempt.getTest().getQuestions()	Test.java → questions List	returns questions without correct_option
ExamPage.jsx	POST /student/exams/answer?attemptId=X	api.js → StudentController.java	{ questionId, selectedOption }
StudentController.java	saveAnswer()	ExamService.java	attemptId, questionId, selectedOption
ExamService.java	compare with question.correctOption	QuestionRepository.java	is_correct evaluation

From File	Calls / Uses	To File	What is passed
ExamService.java	save/update	StudentAnswerRepository.java → MySQL	StudentAnswer row upsert

3.4 Submit Exam & Results

From File	Calls / Uses	To File	What is passed
ExamPage.jsx	POST /student/exams/submit	api.js → StudentController.java	{ attemptId, answers[] }
StudentController.java	submitExam()	ExamService.java	attemptId
ExamService.java	calculates score	(internal)	totalMarks ÷ questionCount × correctAnswers
ExamService.java	save(attempt)	ExamAttemptRepository.java → MySQL	status=COMPLETED, score, end_time, counts
TestReport.jsx	GET /student/exams/attempt/{attemptId}	api.js → StudentController.java	attemptId
TestReport.jsx	GET /student/exams/{attemptId}/answers	api.js → StudentController.java	attemptId
PerformanceReports.jsx	GET /student/performance	api.js → StudentController.java	student ID from JWT

4. Frontend Internal Dependencies

From File	Imports / Uses	To File	Why
App.jsx	imports all page components	All pages under pages/	Route definitions
App.jsx	useAuth()	AuthContext.jsx	PrivateRoute redirect logic
Any page	import Layout	Layout.jsx	Shared nav + wrapper
Any page (logic)	import XxxComponent	components/admin/ or components/student/	Separation of logic and UI
Any page / component	import api	services/api.js	All HTTP calls
services/api.js	reads token	localStorage (set by AuthContext)	Attaches Bearer header
TestManagement.jsx	passes marksPerQuestion prop	TestManagementComponent.jsx	Live marks preview
AvailableTests.jsx	passes completedTestIds prop	AvailableTestsComponent.jsx	Controls badge + button state per test card
Layout.jsx	adminLinks array	(static)	Dashboard, Users, Subjects, Tests, Reports, Profile

5. Backend Internal Dependencies

From File	Injects / Calls	To File	Why
AdminController.java	@Autowired	UserService, SubjectService, TestService, ReportService, ExamService	Delegates all business logic
StudentController.java	@Autowired	ExamService, UserService, TestService, ReportService, EmailService	Student exam operations
AuthController.java	@Autowired	AuthService	Register and login
TestService.java	@Autowired	TestRepository, SubjectRepository, UserRepository, QuestionRepository	Test + question CRUD
TestService.java	accepts DTOs	CreateTestRequest.java, QuestionDTO.java	Inline question creation payload
ExamService.java	@Autowired	ExamAttemptRepository, StudentAnswerRepository, TestRepository, QuestionRepository, UserRepository	Exam lifecycle
AuthService.java	@Autowired	UserRepository, PasswordEncoder, JwtUtil, AuthenticationManager	Auth operations
JwtRequestFilter.java	uses	JwtUtil.java + CustomUserDetailsService.java	Token validation per request
SecurityConfig.java	registers	JwtRequestFilter.java, CustomUserDetailsService.java, PasswordEncoder	Security chain setup
All Repositories	extends JpaRepository	MySQL via Hibernate	ORM — maps Java entities to DB tables